In-Context Engram: An Attention-Free Exact Copy Path for Hybrid Language Models with Less KV Cache

Phung Gia Huy phunghuysoict@gmail.com

Project Proposal — October 2026

1 Problem Description & Current Limitations

Production LLMs increasingly use *hybrid* architectures: about three linear-attention layers (e.g. Gated DeltaNet, GDN) for every full-attention layer, as in Qwen3.5, Qwen3-Next and Kimi Linear. Linear layers keep a fixed-size recurrent state, so only the full-attention layers keep a KV cache that grows with context. At long context that KV cache dominates memory and decode time. The obvious fix, fewer full-attention layers, breaks one ability first: **exact recall of earlier text** (identifiers, numbers, code, quotes). Recurrent models cannot copy strings longer than their state can hold [1], and in hybrids the long-range retrieval is done almost entirely by the full-attention layers [2,3].

Much of that retrieval is *exact copying*. In Zoology [4], 82% of the quality gap between attention and attention-free models on the Pile is explained by in-context recall. Fewer than 5% of heads are "retrieval heads" that copy-paste from context, and removing them breaks retrieval [5]; only these heads need the full KV cache [6]. Yet exact copying needs no learned vectors: if the last n tokens appeared before, the token that followed them is a strong guess for the next token, and a hash table finds it in $O(1)$ from token IDs alone.

Limitation. Current models pay for exact copying with the most expensive resource they have, a per-token KV cache over all past tokens. Existing ways to cut it (KV compression, head-level hybrids [7]) still store vectors and fail first on verbatim recall. **Our thesis:** attention is not bad at copying, it is an *expensive* way to copy.

2 Technical Proposal

2.1 Approach: In-Context Engram (ICE)

Exact lookup (non-learned). At each position t and for each order $n \in \{1, 2, 3, 4, 6, 8\}$, find the most recent $p < t$ with $x_{p-n+1..p} = x_{t-n+1..t}$ and return the *successor* $s_n(t) = x_{p+1}$, the match length and the distance $t - p$. Since $p + 1 \le t$, ICE never sees the future. It stores only token IDs and one hash-table entry per order (≈100 bytes/token, versus ≈2–3 KB/token for a single full-attention layer), and costs $O(1)$ per token. In training, features are computed in the data loader on CPU with a vectorized sort-based routine.

Learned use. ICE gives *hints*, not rules; the model learns how much to trust them:

- *Input side (after layer 2):* $h_t \leftarrow h_t + \sum_n g_n(h_t)\, f_n$, with $f_n = W_n E[s_n(t)] + L[\text{len}] + D[\text{dist}]$ and a context-dependent gate $g_n = \sigma\big(\langle \text{RMSNorm}(h_t), \text{RMSNorm}(f_n)\rangle / \sqrt{d}\big)$.
- *Output side:* a learned copy bonus $\text{logits}_t[s^*(t)] \mathrel{+}= \text{softplus}(w^\top h_t + b_{\text{len}})$ on the longest-match successor $s^*(t)$.
- *ICE-dropout:* ICE is disabled for 20% of sequences, so attention still learns fuzzy retrieval (guarding against "lookup laziness").

Hypotheses. (H1) ICE closes most of the copy-task gap between a 7:1 and a 3:1 hybrid. (H2) 7:1 + ICE matches 3:1 on LM loss. (H3) ICE *adds* to head-level hybrids [7] at equal KV budget, i.e. it is complementary rather than an alternative. (H4, risk) Without ICE-dropout, paraphrased retrieval degrades.

2.2 Evaluation Methods

Step 0 — motivation check (no training). On Qwen3.5-0.8B, compute per-token loss with full attention and with its full-attention layers restricted to a 512-token window. Split the loss increase by whether the ICE successor is correct. If most of the increase falls on ICE-correct tokens, long-range attention is mostly doing copying that ICE can do (go); if not, stop before training.

Main experiment. GDN hybrids (16 layers, d=768, ≈140M parameters, flash-linear-attention), 2.5B tokens (70% FineWeb-Edu, 30% code), 4K context, identical data order and schedule across arms:

Arm	Attention	ICE
A	layer-level 3:1 (4 full-attention layers)	—
B	layer-level 7:1 (2 full-attention layers)	—
C	layer-level 7:1	input + output, dropout 0.2
D	head-level hybrid at the KV budget of B (HydraHead-style [7])	—
E	head-level hybrid at the KV budget of B	input + output, dropout 0.2
F	layer-level 15:1 (1 full-attention layer)	input + output, dropout 0.2

Metrics. (i) Held-out loss split by code/prose and by "copyable" tokens (longest match ≥ 4) vs. the rest; (ii) exact-copy tasks: repeat-string, phonebook, MQAR, including 8K–16K extrapolation; (iii) code completion (RepoBench-P, LCC); (iv) needle retrieval with *templated* and *paraphrased* queries, so that gains from pure string matching are not mistaken for retrieval; (v) retrieval-head formation over training checkpoints; (vi) KV memory per token at matched quality. Two seeds for A–C. **Kill criterion:** C does not beat B on copy tasks beyond seed noise, or ICE hurts paraphrased retrieval and dropout cannot fix it.

2.3 Compute Requests

Throughput estimates assume ≈150–250K tokens/s for a 140M model on one H100 ($\approx 2 \times 10^{18}$ FLOPs, i.e. 3–5 hours per run). To leave spare time, each run is budgeted at **6 H100-hours**, and a 25% contingency covers reruns, crashed jobs and lower-than-expected throughput. Real throughput will be measured on a short run before any long job.

Item	H100-hours	Est. cost (RunPod)
Step 0 motivation check (Qwen3.5-0.8B inference)	6	$15–21
Data download & tokenization (2.5B tokens)	4	$10–14
Arms A–F (6 runs) + second seeds for A–C (3 runs), 6 h each	54	$135–189
Ablations (input-only / output-only ICE, dropout 0), 6 h each	18	$45–63
Evaluation	12	$30–42
Setup & debugging	10	$25–35
Contingency (≈25%: reruns, failures, slower throughput)	26	$65–91
Total	**130**	**$325–455**

Costs assume $2.5–3.5 per H100-hour. Plus ≈300 GB of persistent storage for data and checkpoints. A single GPU suffices; runs are independent, so the work fits in **2–3 weeks** with time left for reruns, or about one week if 2–3 GPUs run in parallel.

3 Brief Literature Review

Copying and recall in efficient models. SSMs fail to copy beyond their state capacity [1]; associative recall explains most of the attention vs. attention-free gap [4]. Induction heads [8] and retrieval heads [5] are the attention circuits that copy; DuoAttention [6] shows only those heads need full KV. **Hybrids.** In GDN/attention hybrids, long-range retrieval lives in the full-attention layers and the recurrent states stay mostly local [2,3]. HydraHead [7] keeps only retrieval-critical *heads* as full attention and matches a 3:1 hybrid at 7:1; it still stores KV vectors, whereas ICE adds a vector-free copy path and is tested as a complement (arm E). **N-gram mechanisms.** The closest work, Akyürek et al. [9], hard-wires "n-gram heads" (attention over hidden states at matched positions) and lowers perplexity by 2.6–6.7% at 340M, even for Transformers; it still needs a per-token vector cache and does not aim to remove attention layers. Engram [10] adds a gated lookup into a *static* n-gram table learned from the corpus; ICE builds its table from the *current context*. Prompt-lookup decoding uses the same matching only to draft tokens at inference. **Novelty:** to our knowledge, no prior work trains an attention-free, token-ID-only in-context n-gram lookup into a hybrid to lower its full-attention ratio.

References

[1] Jelassi et al., *Repeat After Me: Transformers are Better than SSMs at Copying*, arXiv:2402.01032.

[2] *Rethinking the Role of Efficient Attention in Hybrid Architectures*, arXiv:2606.15378.

[3] *Tail-Replay*, arXiv:2608.30310.

[4] Arora et al., *Zoology: Measuring and Improving Recall in Efficient Language Models*, arXiv:2312.04927.

[5] Wu et al., *Retrieval Head Mechanistically Explains Long-Context Factuality*, arXiv:2404.15574.

[6] Xiao et al., *DuoAttention*, arXiv:2410.10819.

[7] Tan et al., *HydraHead: From Head-Level Functional Heterogeneity to Specialized Attention Hybridization*, arXiv:2606.20097.

[8] Olsson et al., *In-context Learning and Induction Heads*, arXiv:2209.11895.

[9] Akyürek et al., *In-Context Language Learning: Architectures and Algorithms*, arXiv:2401.12973.

[10] DeepSeek-AI, *Conditional Memory via Scalable Lookup (Engram)*, arXiv:2601.07372.